Day-94

Hashing.

Map (unique key, value)
- Hash Map — Array of linked List
- Linked Hash Map — Array DLL
- Tree Map — Array Red-black Tree

Set { values } ↑ unique
- Hash Set
- Linked Hash Set
- Tree Set.

Sat → 2:30
Sun → 3:30 — 4:00

key → insertion order maintain

key → Asc order

Any Datatype
Key, value → Any Datatype

Hash Map

insert, remove, search → O(1)

Searching: get(key) → value.
containsKey(key) → True/False

Put(key, value)

remove(key) → o/p : value.

```
HashMap<String, Integer> hm = new HashMap<>();
hm.put("Priyam", 80);
hm.get("  ");
```

keySet

```
map.put(key: "Priyam", value: 80);
map.put(key: "Dhrubo", value: 90);
map.put(key: "Arpan", value: 70);
map.put(key: "Suvajit", value: 65);
```

```
hm.size();
hm.isEmpty();
hm.clear();
```

```
map.keySet();
```

for each loop

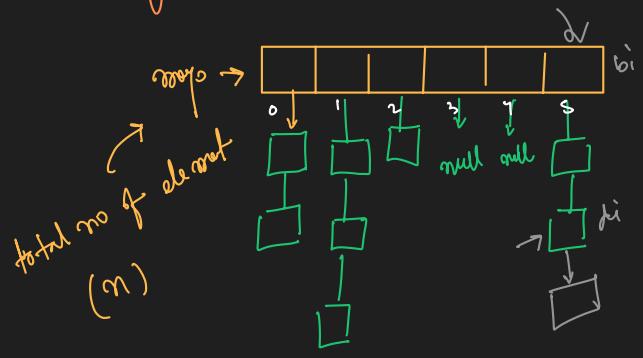
```
for(String key : map.keySet()){
    Sout(key);
}
```

Implementation

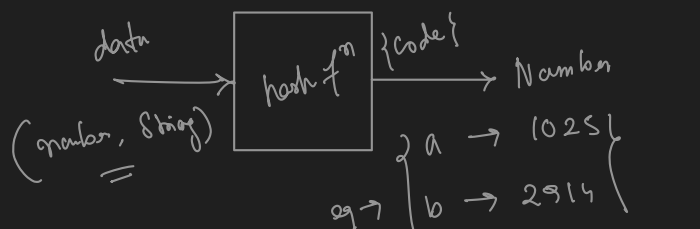
hash map → Array of Linked List

bucket size = $N = 6$

$\lambda = \frac{n}{N}$



hash function → hashCode()



We always have to make sure that $\lambda \leq k$ [threshold value] {2 or 3}

initially $N = 4$, $k = 2$ $\therefore \lambda \leq 2$

$n = 1$, $\lambda = \frac{1}{4} < 2$

$n = 2$, $\lambda = \frac{2}{4} < 2$

⋮

$n = 8$ $\lambda = \frac{8}{4} \leq 2$

$n = 9$, $\lambda = \frac{9}{4} > 2$ → N ↑.

New bucket size = 2 × old bucket size

$[N_1 = 2N]$

$N = 8$ $n = 9$, $\lambda = \frac{9}{8} < 2$

key → hashCode() → 5

priyam → 1011 % N → (0, N-1)

↳ hash function

✓ for a particular bucket size (N)

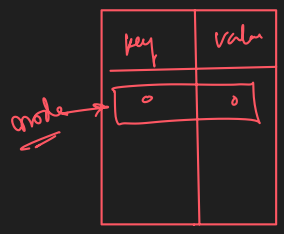
for each key there will be a particular bucket idx (bi)

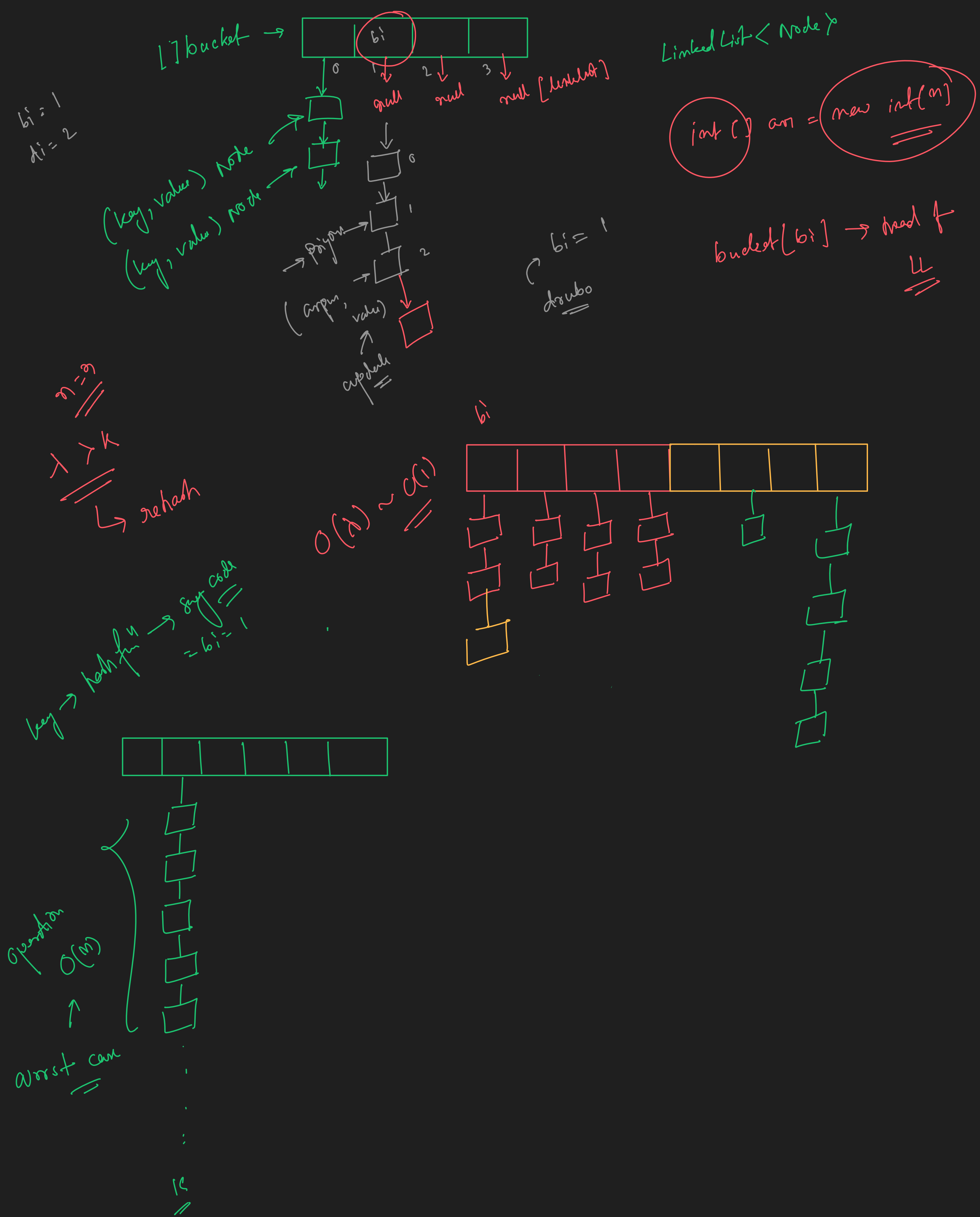
priyam (key) → hash fun → 1011 % N = 1011 % 8 → new bi

HashMap<K, V> map =

↑ ↑

data types





[ ] bucket →
bi
0
1
2
3
null
null
null [LinkedList]
LinkedList<Node>
int [ ] arr = new int[n]
bi = 1
di = 2
(key, value) Node
(key, value) Node
priyam
(arpan, value)
update
bi = 1
dhruba
bucket[bi] → head of
LL
n = 8
λ > K
↳ rehash
O(λ) ~ O(1)
bi
key → hash fun → say code
= bi = 1
Operation
O(n)
worst case
15

```java
static class HashMap<K, V> { // Generic Class

    private class Node {

        K key;
        V value;

        public Node(K key, V value) {
            this.key = key;
            this.value = value;
        }
    }

    private int N; // Bucket Size
    private int n; // total no. of element is the map

    private LinkedList<Node> bucket[]; // N

    @SuppressWarnings("unchecked")
    public HashMap(){
        this.n = 0;
        this.N = 4;
        this.bucket = new LinkedList[N];

        for(int i=0; i<bucket.length; i++){
            bucket[i] = new LinkedList<>();
        }
    }

    public void put(K key, V value){
        int bi = hashFuncton(key); // 0 to N-1
        int di = serachINLL(key, bi);

        if(di != -1){
            // update
            Node node = bucket[bi].get(di);
            node.value = value;

        }else{ // -1
            // add new node
            bucket[bi].add(new Node(key, value));
            n++;
        }

        double lambda = (double)n/N;
        if(lambda > 2.0){
            rehash();
        }
    }

    public boolean containsKey(K key){
        int bi = hashFuncton(key);
        int di = serachINLL(key, bi);

        if(di != -1){
            return true;
        }else{
            return false;
        }
    }

    public V get(K key){
        int bi = hashFuncton(key);
        int di = serachINLL(key, bi);

        if(di != -1){
            Node node = bucket[bi].get(di);
            return node.value;
        }else{
            return null;
        }
    }

    public V remove(K key){
        int bi = hashFuncton(key);
        int di = serachINLL(key, bi);

        if(di != -1){
            Node node = bucket[bi].remove(di);
            n--;
            return node.value;
        }else{
            return null;
        }
    }

    private int hashFuncton(K key){
        return Math.abs(key.hashCode()) % N;
    }

    private int serachINLL(K key, int bi){
        LinkedList<Node> ll = bucket[bi];

        for(int i=0; i<ll.size(); i++){
            Node node = ll.get(i);
            if(node.key == key){
                return i;
            }
        }

        return -1;
    }

    @SuppressWarnings("unchecked")
    private void rehash(){
        LinkedList<Node>[] oldBucket = bucket;

        bucket = new LinkedList[2*N]; // new bucket
        N = 2*N;

        for(int i=0; i<bucket.length; i++){
            bucket[i] = new LinkedList<>();
        }

        for(int i=0; i<oldBucket.length; i++){
            LinkedList<Node> ll = oldBucket[i];

            for(int j=0; j<ll.size(); j++){
                Node node = ll.get(j);
                put(node.key, node.value);
            }
        }
    }
```

```java
    public boolean isEmpty(){
        return n==0;
    }

    public List<K> keySet(){
        List<K> keys = new ArrayList<>();

        for(int i=0; i<bucket.length; i++){
            LinkedList<Node> ll = bucket[i];
            for (Node node : ll) {
                keys.add(node.key);
            }
        }

        return keys;
    }
}
```